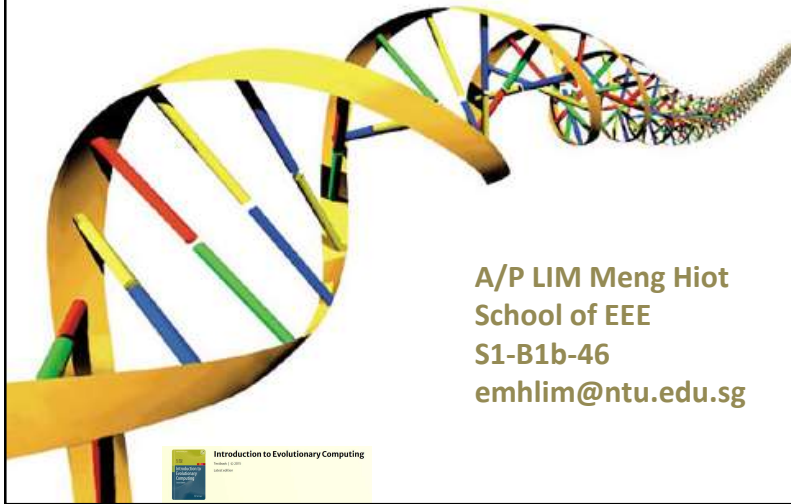


EE6407: Genetic Algorithms and Machine Learning



A/P LIM Meng Hiot
School of EEE
S1-B1b-46
emhlim@ntu.edu.sg

1

Assessment

- Continuous Assessment – 40%
- Final Exam (3-hour) – 60%

Continuous Assessment (Part 1 – 4 Weeks):

Quiz – 10%

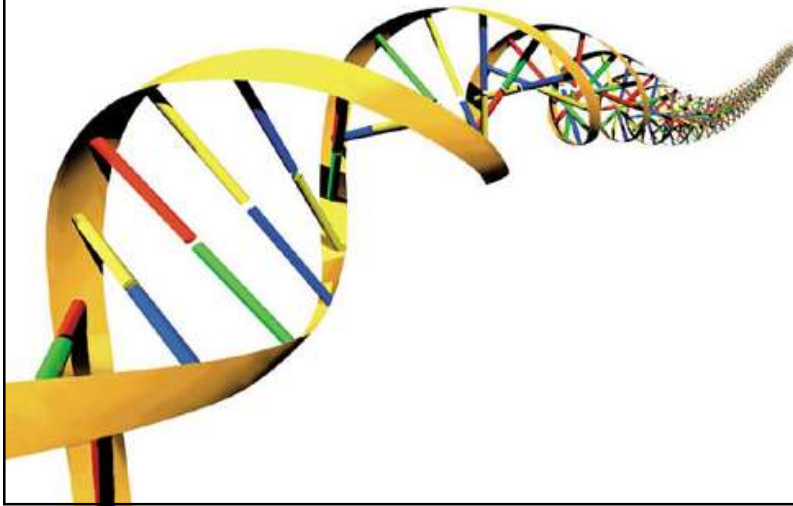
When? – Last hour of Week 4

2

2

1

Evolutionary Computing



3

Problems to be solved

Problems can be classified in different ways:

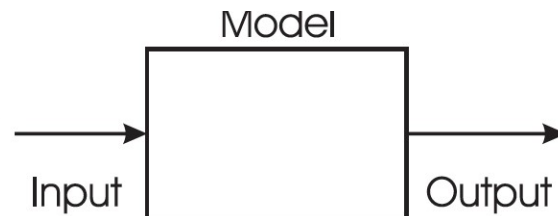
- Black box model
- Search problems
- Optimization vs constraint satisfaction
- NP problems

4

4

2

“Black box” model



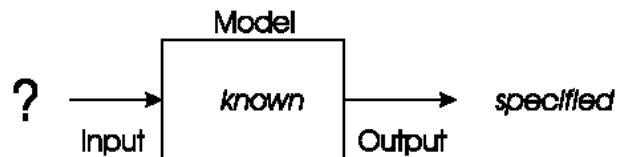
- “Black box” model consists of 3 components
- When one component is unknown: new problem type

5

5

“Black box” model: Optimisation

- Model and desired output is known, task is to find inputs



- Examples:
 - Timetables for university, call center, or hospital
 - Design specifications
 - Traveling salesman problem (TSP)
 - Eight-queens problem, etc.

6

6

3

“Black box” model: Optimisation example 1: university timetabling

- Enormously big search space
- Timetables must be *good*
- “Good” is defined by a number of competing criteria
- Timetables must be feasible
- Vast majority of search space is infeasible

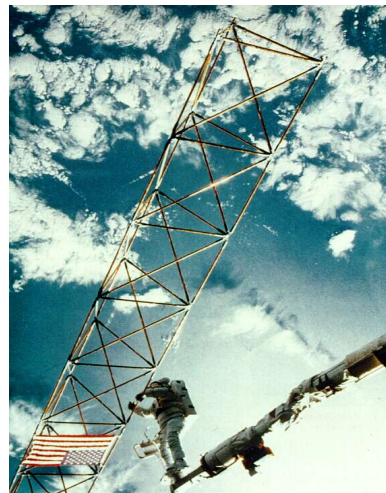


7

7

“Black box” model: Optimisation example 2: satellite structure

- Optimised satellite designs for NASA to maximize vibration isolation
- Evolving: design structures
- Fitness: vibration resistance
- Evolutionary “creativity”



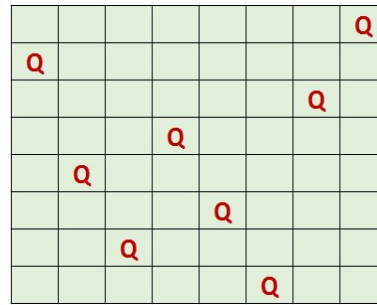
8

8

4

“Black box” model: Optimisation example 3: 8 queens problem

- Given an 8-by-8 chessboard and 8 queens
- Place the 8 queens on the chessboard without any conflict
- Two queens in conflict if they share same row, column or diagonal
- Can be extended to an n queens problem ($n > 8$)

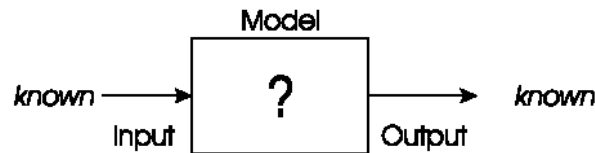


9

9

“Black box” model: Modelling

- We have corresponding sets of inputs & outputs and seek model that delivers correct output for every known input



- Note: modelling problems can be transformed into optimisation problems
 - Evolutionary machine learning
 - Predicting stock exchange
 - Voice control system for smart homes

10

10

“Black box” model: Modelling example: loan applicant creditability

- Creditability model to predict loan paying behavior of new applicants
- Evolving: prediction models
- Fitness: model accuracy on historical data

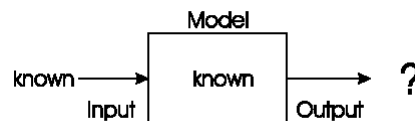


11

11

“Black box” model: Simulation

- We have a given model and wish to know the outputs that arise under different input conditions

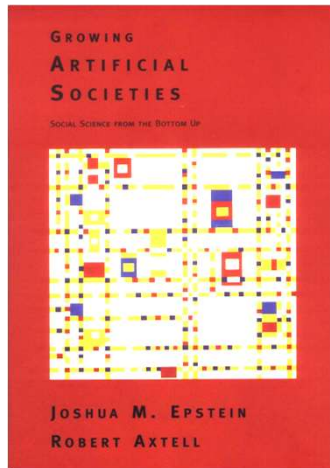


- Often used to answer “what-if” questions in evolving dynamic environments
 - Evolutionary economics, Artificial Life
 - Weather forecast system
 - Impact analysis new tax systems

12

12

“Black box” model: Simulation example: evolving artificial societies



Simulating trade, economic competition, etc. to **calibrate models**

Use models to optimise strategies and policies

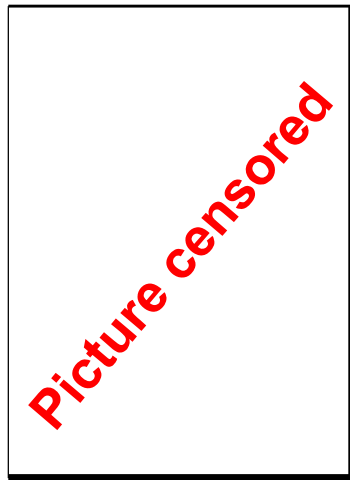
Evolutionary economy

Survival of the fittest is universal (big/small fish)

11

13

“Black box” model: Simulation example 2: biological interpretations



Incest prevention keeps evolution from rapid degeneration
(we knew this)

Multi-parent reproduction, makes evolution more efficient
(this does not exist on Earth in carbon)

2nd sample of Life

14

14

Search problems

- Simulation is different from optimisation/modelling
- Optimisation/modelling problems search through huge space of possibilities
- Search space: collection of all objects of interest including the desired solution
- **Question:** how large is the search space for different tours through n cities?

Benefit of classifying these problems: distinction between

- search problems, which define search spaces, and
- problem-solvers, which tell how to move through search spaces.

15

15

Optimisation vs. constraint satisfaction (1/2)

- Objective function: a way of assigning a value to a possible solution that reflects its quality on scale
 - Number of un-checked queens (maximize)
 - Length of a tour visiting given set of cities (minimize)
- Constraint: binary evaluation telling whether a given requirement holds or not
 - Find a configuration of eight queens on a chessboard such that no two queens check each other
 - Find a tour with minimal length where city X is visited after city Y

16

16

Optimisation vs. constraint satisfaction (2/2)

- When combining the two:

Constraints	Objective function	
	Yes	No
Yes	Constrained optimisation problem	Constraint satisfaction problem
No	Free optimisation problem	No problem

- Where do the examples fit?
- Note: constraint problems can be transformed into optimisation problems
- Question: how can we formulate the 8-queens problem into a FOP/CSP/COP?

17

17

NP problems

- We only looked at classifying the problem, not discussed problem solvers
- This classification scheme needs the properties of the problem solver
- Benefit of this scheme: possible to tell how difficult the problem is
- Explain the basics of this classifier for combinatorial optimization problems (Booleans or integers search space)

18

18

NP problems: Key notions

- **Problem size**: dimensionality of the problem at hand and number of different values for the problem variables
- **Running-time**: number of operations the algorithm takes to terminate
 - Worst-case as a function of problem size
 - Polynomial, super-polynomial, exponential
- **Problem reduction**: transforming current problem into another via mapping

19

19

NP problems: Class

- The difficultness of a problem can now be classified:
 - **Class P**: algorithm can solve the problem in polynomial time (worst-case running-time for problem size n is less than $F(n)$ for some polynomial formula F)
 - **Class NP**: problem can be solved, and any solution can be verified within polynomial time by some other algorithm (P subset of NP)
 - **Class NP-complete**: problem belongs to class NP and any other problem in NP can be reduced to this problem by an algorithm running in polynomial time
 - **Class NP-hard**: problem is at least as hard as any other problem in NP -complete but solution cannot necessarily be verified within polynomial time

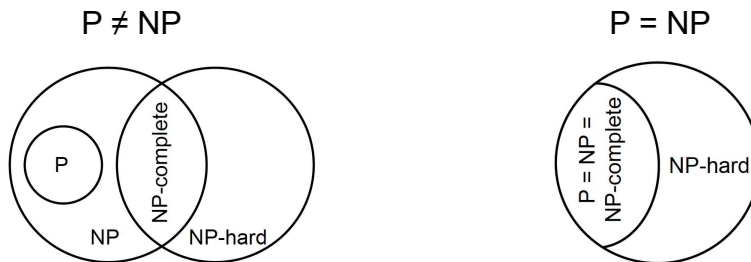
20

20

10

NP problems: Difference between classes

- P is different from NP-hard
- Not known whether P is different from NP



- For now: use of approximation algorithms and metaheuristics

21

21

Computers, Complexity, and Intractability

One day your boss calls you into his office and confides that the company is about to enter the highly competitive “bandersnatch” market. For this reason, a good method is needed for determining whether or not any given set of specifications for a new bandersnatch component can be met and, if so, for constructing a design that meets them. Since you are the company’s chief algorithm designer, your charge is to find an efficient algorithm for doing this.

After consulting with the bandersnatch department to determine exactly what the problem is, you eagerly hurry back to your office, pull down your reference books, and plunge into the task with great enthusiasm. Some weeks later, your office filled with mountains of crumpled-up scratch paper, your enthusiasm has lessened considerably. So far you have not been able to come up with any algorithm substantially better than searching through all possible designs. This would not particularly endear you to your boss, since it would involve years of computation time for just one set of

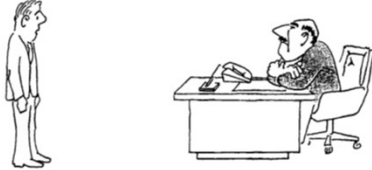
22

22

11

2 COMPUTERS, COMPLEXITY, AND INTRACTABILITY

specifications, and the bandersnatch department is already 13 components behind schedule. You certainly don't want to return to his office and report:



"I can't find an efficient algorithm, I guess I'm just too dumb."

To avoid serious damage to your position within the company, it would be much better if you could prove that the bandersnatch problem is *inherently* intractable, that no algorithm could possibly solve it quickly. You then could stride confidently into the boss's office and proclaim:



"I can't find an efficient algorithm, because no such algorithm is possible!"

Unfortunately, proving inherent intractability can be just as hard as finding efficient algorithms. Even the best theoreticians have been stymied in their attempts to obtain such proofs for commonly encountered hard problems. However, having read this book, you have discovered something

1.1 INTRODUCTION 3

almost as good. The theory of NP-completeness provides many straightforward techniques for proving that a given problem is "just as hard" as a large number of other problems that are widely recognized as being difficult and that have been confounding the experts for years. Armed with these techniques, you might be able to prove that the bandersnatch problem is NP-complete and, hence, that it is equivalent to all these other hard problems. Then you could march into your boss's office and announce:



"I can't find an efficient algorithm, but neither can all these famous people."

At the very least, this would inform your boss that it would do no good to fire you and hire another expert on algorithms.

Of course, our own bosses would frown upon our writing this book if its sole purpose was to protect the jobs of algorithm designers. Indeed, discovering that a problem is NP-complete is usually just the beginning of work on that problem. The needs of the bandersnatch department won't disappear overnight simply because their problem is known to be NP-complete. However, the knowledge that it is NP-complete does provide valuable information about what lines of approach have the potential of being most productive. Certainly the search for an efficient, exact algorithm should be accorded low priority. It is now more appropriate to concentrate on other, less ambitious, approaches. For example, you might look for efficient algorithms that solve various special cases of the general problem. You might look for algorithms that, though not guaranteed to run quickly, seem likely to do so most of the time. Or you might even relax the problem somewhat, looking for a fast algorithm that merely finds designs that

23

Evolutionary Computing: the Origins

- Historical perspective
- Biological inspiration:
 - Darwinian evolution theory (simplified!)
 - Genetics (simplified!)
- Motivation for EC

24

24

12

Historical perspective

- 1948, Turing:
proposes “genetical or evolutionary search”
- 1962, Bremermann:
optimization through evolution and recombination
- 1964, Rechenberg:
introduces evolution strategies
- 1965, L. Fogel, Owens and Walsh:
introduce evolutionary programming
- 1975, Holland:
introduces genetic algorithms
- 1992, Koza:
introduces genetic programming

25

25

Darwinian Evolution (1/3): Survival of the fittest

Evolutionary Computing: the Origins

- Historical perspective
- Biological inspiration:
 - Darwinian evolution theory (simplified)
 - Genetics (simplified)
- Motivation for EC

- All environments have finite resources
(i.e., can only support a limited number of individuals)
- Life forms have basic instinct/ lifecycles geared towards reproduction
- Therefore, some kind of selection is inevitable
- Those individuals that compete for the resources most effectively have increased chance of reproduction
- Note: fitness in natural evolution is a derived, secondary measure, i.e., we (humans) assign a high fitness to individuals with many offspring

26

26

Darwinian Evolution (2/3): Diversity drives change

- Phenotypic traits:
 - Behaviour / physical differences that affect response to environment
 - Partly determined by inheritance, partly by factors during development
 - Unique to each individual, partly as a result of random changes
- If phenotypic traits:
 - Lead to higher chances of reproduction
 - Can be inherited
 then they will tend to increase in subsequent generations, leading to new combinations of traits ...

27

27

Darwinian Evolution (3/3): Summary

- Population consists of diverse set of individuals
- Combinations of traits that are better adapted tend to increase representation in population

Individuals are “units of selection”
- Variations occur through random changes yielding constant source of diversity, coupled with selection means that:

Population is the “unit of evolution”

28

28

Genetics: Natural

Evolutionary Computing: the Origins

- Historical perspective
- Biological inspiration:
 - Darwinian evolution theory (simplified!)
 - Genetics (simplified!)
- Motivation for EC

- The information required to build a living organism is coded in the DNA of that organism
- Genotype (DNA inside) determines phenotype
- Genes → phenotypic traits is a complex mapping
 - One gene may affect many traits (pleiotropy)
 - Many genes may affect one trait (polygeny)
- Small changes in the genotype lead to small changes in the organism (e.g., height, hair colour)

29

29

Genetics: Genes and the Genome

- Genes are encoded in strands of DNA called chromosomes
- In most cells, there are two copies of each chromosome (diploidy)
- The complete genetic material in an individual's genotype is called the Genome
- Within a species, most of the genetic material is the same

30

30

Genetics: Mutation

- Occasionally some of the genetic material changes very slightly during this process (replication error)
- This means that the child might have genetic material information not inherited from either parent
- This can be
 - catastrophic: offspring is not viable (most likely)
 - neutral: new feature not influences fitness
 - advantageous: strong new feature occurs
- Redundancy in the genetic code forms a good way of error checking

31

31

Recap of EC metaphor (1/2)

Evolutionary Computing: the Origins

- Historical perspective
- Biological inspiration:
 - Darwinian evolution theory (simplified!)
 - Genetics (simplified!)
- Motivation for EC

- A population of individuals exists in an environment with limited resources
- **Competition** for those resources causes selection of those **fitter** individuals that are better adapted to the environment
- These individuals act as seeds for the generation of new individuals through recombination and mutation
- The new individuals have their fitness evaluated and compete (possibly also with parents) for survival.
- Over time **Natural selection** causes a rise in the fitness of the population

32

32

Recap of EC metaphor (2/2)

- EAs fall into the category of “generate and test” algorithms
- They are stochastic, population-based algorithms
- Variation operators (recombination and mutation) create the necessary diversity and thereby facilitate novelty
- Selection reduces diversity and acts as a force pushing quality

33

33

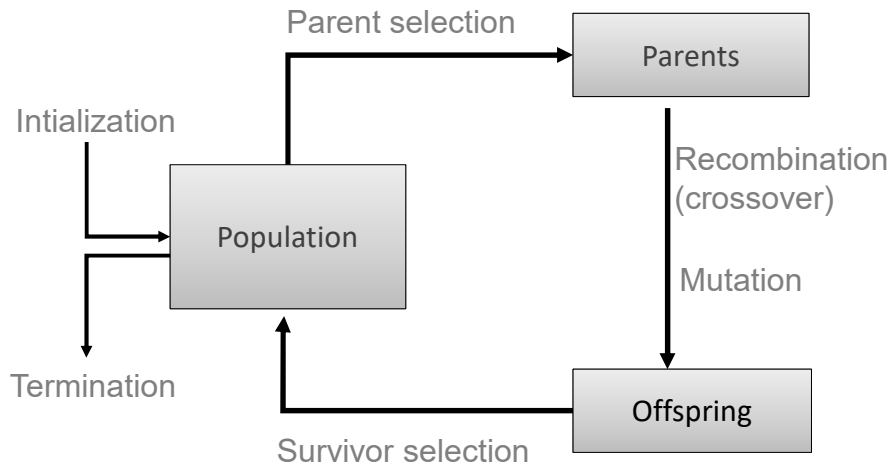
What is an Evolutionary Algorithm?

- Scheme of an EA
- Main EA components:
 - Representation / evaluation / population
 - Parent selection / survivor selection
 - Recombination / mutation
- Examples: eight-queens problem
- Typical EA behaviour
- EAs and global optimisation
- EC and neighbourhood search

34

34

Scheme of an EA: General scheme of EAs



35

35

Scheme of an EA: EA scheme in pseudo-code

```

BEGIN
  INITIALISE population with random candidate solutions;
  EVALUATE each candidate;
  REPEAT UNTIL ( TERMINATION CONDITION is satisfied ) DO
    1 SELECT parents;
    2 RECOMBINE pairs of parents;
    3 MUTATE the resulting offspring;
    4 EVALUATE new candidates;
    5 SELECT individuals for the next generation;
  OD
END
  
```

36

36

Scheme of an EA: Common model of evolutionary processes

- **Population** of individuals
- Individuals have a **fitness**
- **Variation** operators: crossover, mutation
- **Selection** towards higher fitness
 - “survival of the fittest” and
 - “mating of the fittest”

Neo Darwinism:

Evolutionary progress towards higher life forms

=

Optimization according to some fitness-criterion
(optimization on a fitness landscape)

37

37

Scheme of an EA: Two pillars of evolution

There are two competing forces

Increasing population **diversity**
by genetic operators

- mutation
- recombination

Push towards **novelty**

Decreasing population **diversity** by
selection

- of parents
- of survivors

Push towards **quality**

38

38

19

Main EA components: Representation (1/2)

What is an Evolutionary Algorithm?

- Scheme of an EA
- Main EA components:
 - Representation / evaluation / population
 - Parent selection / survivor selection
 - Recombination / mutation
- Examples: eight-queens problem
- Typical EA behaviour
- EAs and global optimisation
- EC and neighbourhood search

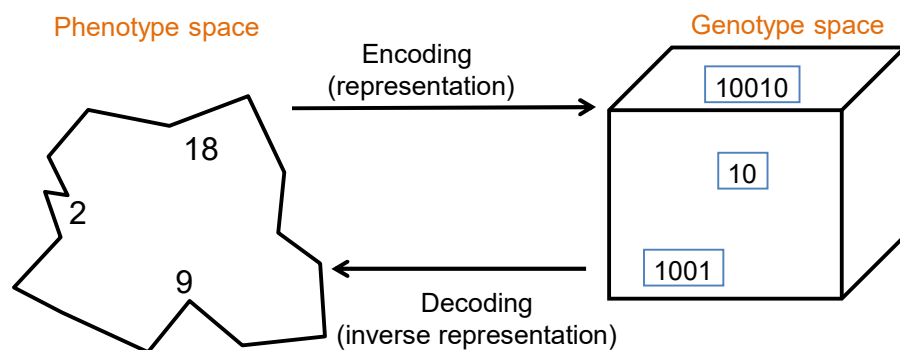
- Role: provides code for candidate solutions that can be manipulated by variation operators
- Leads to two levels of existence
 - **phenotype**: **object** in original problem context, the outside
 - **genotype**: **code** to denote that object, the inside (chromosome, “digital DNA”)
- Implies two mappings:
 - Encoding : phenotype=> genotype (not necessarily one to one)
 - Decoding : genotype=> phenotype (must be one to one)
- Chromosomes contain **genes**, which are in (usually fixed) positions called **loci** (sing. locus) and have a value (**allele**)

39

39

Main EA components: Representation (2/2)

Example: represent integer values by their binary code



In order to find the global optimum, every feasible solution must be represented in genotype space

40

40

20

Main EA components: Evaluation (fitness) function

What is an Evolutionary Algorithm?

- Scheme of an EA
- Main EA components:
 - Representation / evaluation / population
 - Parent selection / survivor selection
 - Recombination / mutation
- Examples: eight-queens problem
- Typical EA behaviour
- EAs and global optimisation
- EC and neighbourhood search

- Role:
 - Represents the task to solve, the requirements to adapt to (can be seen as “the environment”)
 - Enables selection (provides basis for comparison)
 - e.g., some phenotypic traits are advantageous, desirable, e.g. big ears cool better, these traits are rewarded by more offspring that will expectedly carry the same trait
- A.k.a. *quality* function or *objective* function
- Assigns a single real-valued fitness to each phenotype which forms the basis for selection
 - So the more discrimination (different values) the better
- Typically we talk about fitness being maximised
 - Some problems may be best posed as minimisation problems, but conversion is trivial

41

41

Main EA components: Population (1/2)

What is an Evolutionary Algorithm?

- Scheme of an EA
- Main EA components:
 - Representation / evaluation / population
 - Parent selection / survivor selection
 - Recombination / mutation
- Examples: eight-queens problem
- Typical EA behaviour
- EAs and global optimisation
- EC and neighbourhood search

- Role: holds the candidate solutions of the problem as individuals (genotypes)
- Formally, a population is a multiset of individuals, i.e. repetitions are possible
- Population is the basic unit of evolution, i.e., the population is evolving, not the individuals
- Selection operators act on population level
- Variation operators act on individual level

42

42

Main EA components: Population (2/2)

- Some sophisticated EAs also assert a spatial structure on the population e.g., a grid
- Selection operators usually take whole population into account i.e., reproductive probabilities are *relative* to *current* generation
- **Diversity** of a population refers to the number of different fitnesses / phenotypes / genotypes present (note: not the same thing)

43

43

Main EA components: Selection mechanism (1/3)

Role:

- Identifies individuals
 - to become parents
 - to survive
- Pushes population towards higher fitness
- Usually probabilistic
 - high quality solutions more likely to be selected than low quality
 - but not guaranteed
 - even worst in current population usually has non-zero probability of being selected
- This *stochastic* nature can aid escape from local optima

What is an Evolutionary Algorithm?

- Scheme of an EA
- Main EA components:
 - Representation / evaluation / population
 - Parent selection / survivor selection
 - Recombination / mutation
- Examples: eight-queens problem
- Typical EA behaviour
- EAs and global optimisation
- EC and neighbourhood search

44

44

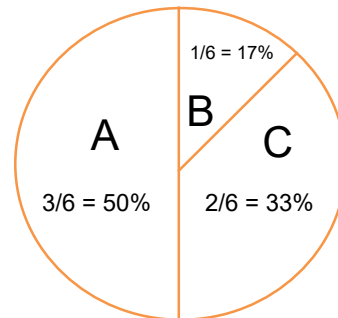
Main EA components: Selection mechanism (2/3)

Example: roulette wheel selection

fitness(A) = 3

fitness(B) = 1

fitness(C) = 2



In principle, any selection mechanism can be used for parent selection as well as for survivor selection

45

45

Main EA components: Selection mechanism (3/3)

- Survivor selection A.k.a. **replacement**
- Most EAs use fixed population size so need a way of going from (parents + offspring) to next generation
- Often deterministic (while parent selection is usually stochastic)
 - Fitness based : e.g., rank parents + offspring and take best
 - Age based: make as many offspring as parents and delete all parents
- Sometimes a combination of stochastic and deterministic (elitism)

46

46

Main EA components: Variation operators

What is an Evolutionary Algorithm?

- Scheme of an EA
- Main EA components:
 - Representation / evaluation / population
 - Parent selection / survivor selection
 - Recombination / mutation
- Examples: eight-queens problem
- Typical EA behaviour
- EAs and global optimisation
- EC and neighbourhood search

- Role: to generate new candidate solutions
- Usually divided into two types according to their **arity** (number of inputs):
 - Arity 1 : mutation operators
 - Arity >1 : recombination operators
 - Arity = 2 typically called **crossover**
 - Arity > 2 is formally possible, seldom used in EC
- There has been much debate about relative importance of recombination and mutation
 - Nowadays most EAs use both
 - Variation operators must match the given representation

47

47

Main EA components: Mutation (1/2)

- Role: causes small, random variance
- Acts on one genotype and delivers another
- Element of randomness is essential and differentiates it from other unary heuristic operators
- Importance ascribed depends on representation and historical dialect:
 - Binary GAs – background operator responsible for preserving and introducing diversity
 - EP for FSM's / continuous variables – only search operator
 - GP – hardly used
- May guarantee connectedness of search space and hence convergence proofs

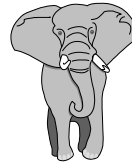
48

48

Main EA components: Mutation (2/2)

before

1 1 1 1 1 1 1



after

1 1 1 0 1 1 1



49

49

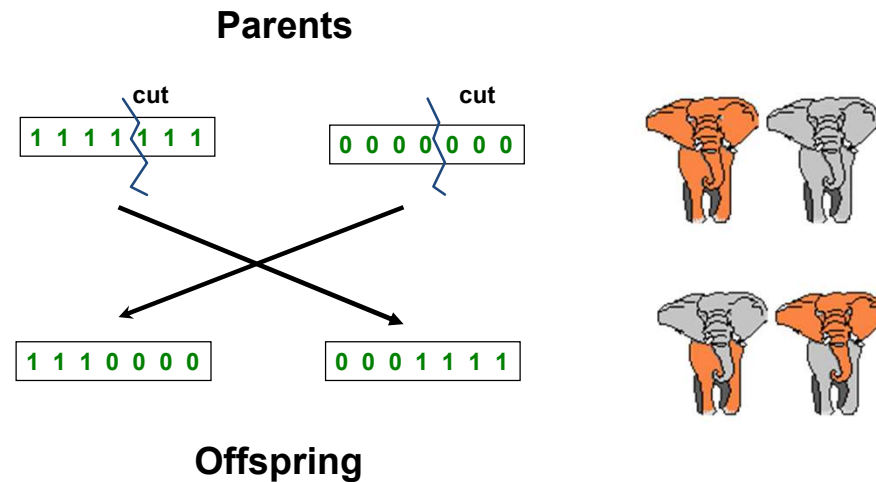
Main EA components: Recombination (1/2)

- Role: merges information from parents into offspring
- Choice of what information to merge is stochastic
- Most offspring may be worse, or the same as the parents
- Hope is that some are better by combining elements of genotypes that lead to good traits
- Principle has been used for millennia by breeders of plants and livestock

50

50

Main EA components: Recombination (2/2)



51

51

Main EA components: Initialisation / Termination

What is an Evolutionary Algorithm?

- Scheme of an EA
- Main EA components:
 - Representation / evaluation / population
 - Parent selection / survivor selection
 - Recombination / mutation
- Examples: eight-queens problem
- Typical EA behaviour
- EAs and global optimisation
- EC and neighbourhood search

- Initialisation usually done at random,
 - Need to ensure even spread and mixture of possible allele values
 - Can include existing solutions, or use problem-specific heuristics, to “seed” the population
- Termination condition checked every generation
 - Reaching some (known/hoped for) fitness
 - Reaching some maximum allowed number of generations
 - Reaching some minimum level of diversity
 - Reaching some specified number of generations without fitness improvement

52

52

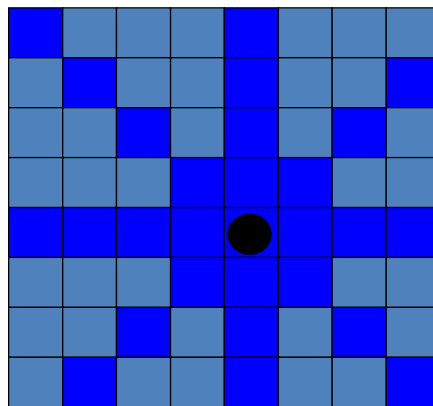
Main EA components: What are the different types of EAs

- Historically different flavours of EAs have been associated with different data types to represent solutions
 - Binary strings : Genetic Algorithms
 - Real-valued vectors : Evolution Strategies
 - Finite state Machines: Evolutionary Programming
 - LISP trees: Genetic Programming
- These differences are largely irrelevant, best strategy
 - choose representation to suit problem
 - choose variation operators to suit representation
- Selection operators only use fitness and so are independent of representation

53

53

Example: The 8-queens problem



Place 8 queens on an 8x8 chessboard in such a way that they cannot check each other

54

54

What is an Evolutionary Algorithm?

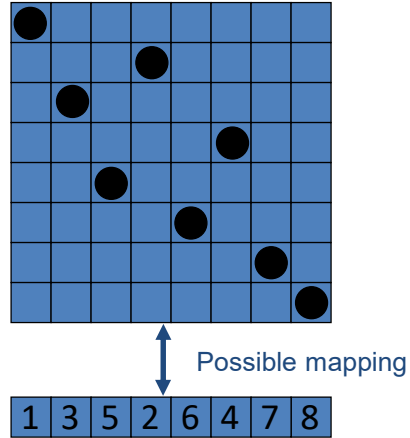
- Scheme of an EA
- Main EA components:
 - Representation / evaluation / population
 - Parent selection / survivor selection
 - Recombination / mutation
- Examples: eight-queens problem
- Typical EA behaviour
- EAs and global optimisation
- EC and neighbourhood search

27

The 8-queens problem: Representation

Phenotype:
a board configuration

Genotype:
a permutation of
the numbers 1–8



55

55

The 8-queens problem: Fitness evaluation

- **Penalty** of one queen: the number of queens she can check
- Penalty of a configuration: the sum of penalties of all queens
- Note: penalty is to be minimized
- **Fitness** of a configuration: inverse penalty to be maximized

56

56

The 8-queens problem: Mutation

Small variation in one permutation, e.g.:

- swapping values of two randomly chosen positions,



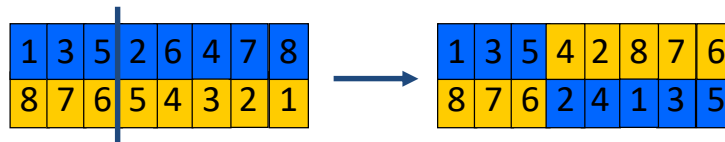
57

57

The 8-queens problem: Recombination

Combining two permutations into two new permutations:

- choose random crossover point
- copy first parts into children
- create second part by inserting values from other parent:
 - in the order they appear there
 - beginning after crossover point
 - skipping values already in child



58

58

The 8-queens problem: Selection

- Parent selection:
 - Pick 5 parents and take best two to undergo crossover
- Survivor selection (replacement)
 - When inserting a new child into the population, choose an existing member to replace by:
 - sorting the whole population by decreasing fitness
 - enumerating this list from high to low
 - replacing the first with a fitness lower than the given child

59

59

The 8-queens problem: Summary

Representation	Permutations
Recombination	“Cut-and-crossfill” crossover
Recombination probability	100%
Mutation	Swap
Mutation probability	80%
Parent selection	Best 2 out of random 5
Survival selection	Replace worst
Population size	100
Number of Offspring	2
Initialisation	Random
Termination condition	Solution or 10,000 fitness evaluation

Note that is **only one possible**
set of choices of operators and parameters

60

60

SGA Example: $f(x)=x^2$

What is an Evolutionary Algorithm?

- Scheme of an EA
- Main EA components:
 - Representation / evaluation / population
 - Parent selection / survivor selection
 - Recombination / mutation
- Examples: eight-queens problem
- Typical EA behaviour
- EAs and global optimisation
- EC and neighbourhood search

- Consider a problem of maximizing $f(x)=x^2$.
- Assume x is an integer between 0 and 31.
- Go through with more details and highlight the 6 major steps.

61

61

SGA – Steps

Step 1 (Encoding)

- In this simple problem, we have just one integer variable restricted between 0 and 31.
- One obvious encoding is to use 5 bits.

Step 2 (Initial population)

- The initial population is generated randomly.

Step 3 (Decoding)

- The strings are converted to integers and $f(x)$ are evaluated.
- The (initial) population, decoded decision variable x , fitness $f(x)$, fitness proportions are all shown in Table 1.2.

Step 4 (Computing fitness proportions)

- We can use the roulette wheel to obtain the mating pool (shown in column 7 in Table 1.2)

62

62

SGA – Steps...

Step 5 (Crossover)

- The 1st step is to select $n/2$ pairs of parent strings for crossover. The randomly chosen pairs are shown in column 8 under “Mate”.
- Then the one-point crossover sites are randomly chosen between 1 and 4. It is shown in column 9 as “C’over site”.
- The result of crossover operation is given in column 10, as “New Popn”.

Step 6

- Here we assume that the mutation probability is 0.001. There are 20 bits in total in the population. Hence, 0.02 bits should mutate.
- Hence, no bit mutate, and the final result (next generation population) is shown in column 10.

63

63

Evolution & Termination of the SGA

- In general, steps 1 & 2 performed just once in the beginning of the SGA.
- Steps 3, 4, 5 and 6 are repeated until a termination condition is satisfied.
- Possible termination conditions may be
 - A given number of generations (or a number of fitness evaluations) has been completed
 - The fitness (i.e. the objective value of the optimization problem) of the best solution or the pool has reached an acceptable level.
 - The sequential increase in the fitness is negligible over several generations.
 - A combination of the above conditions.

64

64

String No.	Initial Popn.	x	$f(x)$	% of Total	No. Sel.	Mating Pool	Mate	C'over Site	New Popn.	x	$f(x)$
1	01101	13	169	14.4	1	01101	2	4	01100	12	144
2	11000	24	576	49.2	2	11000	1	4	11001	25	625
3	01000	8	64	5.5	0	11000	4	2	11011	27	729
4	10011	19	361	30.9	1	10011	3	2	10000	16	256
Sum			1170	100.0	4						1754
Average			293								439

Table 1.2 Simulation of a simple GA

65

65

String No.	Initial Popn.	x	$f(x)$	% of Total	No. Sel.	Mating Pool	Mate	C'over Site	New Popn.	x	$f(x)$
1	01100	12	144				2	4			
2	11001	25	625				1	4			
3	11011	27	729				4	2			
4	10000	16	256				3	2			
Sum				100.0	4						
Average											

Table 1.2 Simulation of a simple GA

66

66

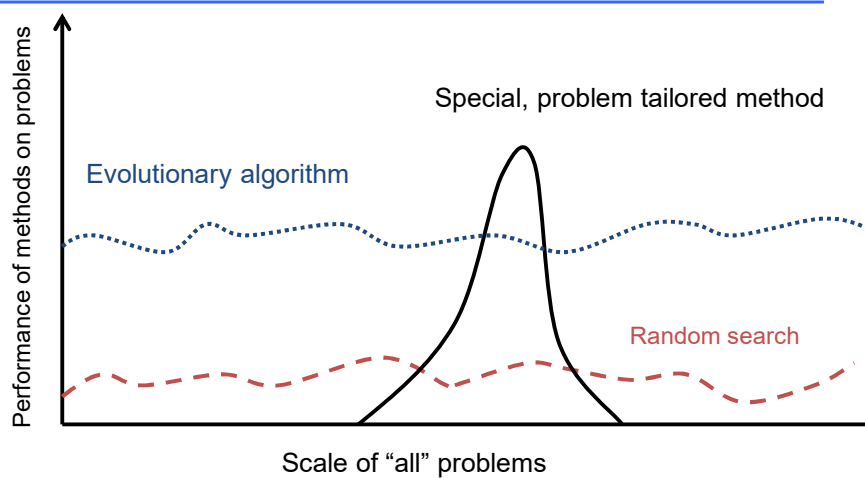
Typical EA behaviour: Evolutionary Algorithms in context

- There are many views on the use of EAs as robust problem-solving tools
- For most problems, a problem-specific tool may:
 - perform better than a generic search algorithm on most instances,
 - have limited utility,
 - not do well on all instances
- Goal is to provide robust tools that provide:
 - evenly good performance
 - over a range of problems and instances

67

67

Typical EA behaviour: EAs as problem solvers: Goldberg view (1989)



68

68

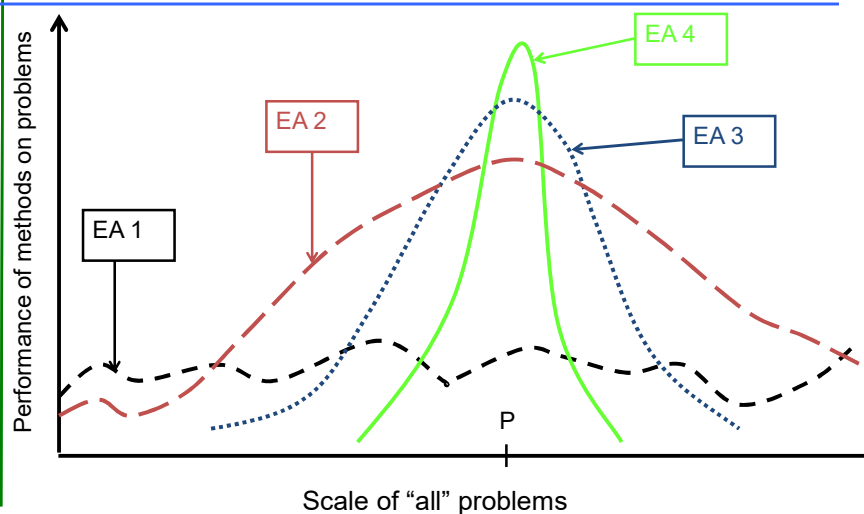
Typical EA behaviour: EAs and domain knowledge

- Trend in the 90's:
adding problem specific knowledge to EAs
(special variation operators, repair, etc)
- Result: EA performance curve "deformation":
 - better on problems of the given type
 - worse on problems different from given type
 - amount of added knowledge is variable
- Recent theory suggests the search for an "all-purpose" algorithm may be fruitless

69

69

Typical EA behaviour: EAs as problem solvers: Michalewicz view (1996)



70

70

EC and global optimisation

- Global Optimisation: search for finding best solution x^* out of some fixed set S
- Deterministic approaches
 - e.g. box decomposition (branch and bound etc)
 - Guarantee to find x^* ,
 - May have bounds on runtime, usually super-polynomial
- Heuristic Approaches (generate and test)
 - rules for deciding which $x \in S$ to generate next
 - no guarantees that best solutions found are globally optimal
 - no bounds on runtime
- “I don’t care if it works as long as it converges”
- vs.
- “I don’t care if it converges as long as it works”

71

71

EC and neighbourhood search

What is an Evolutionary Algorithm?

- Scheme of an EA
- Main EA components:
 - Representation / evaluation / population
 - Parent selection / survivor selection
 - Recombination / mutation
- Examples: eight-queens problem
- Typical EA behaviour
- EAs and global optimisation
- EC and neighbourhood search

- Many heuristics impose a neighbourhood structure on S
- Such heuristics may guarantee that best point found is *locally optimal* e.g. Hill-Climbers:
 - **But** problems often exhibit many local optima
 - Often very quick to identify good solutions
- EAs are distinguished by:
 - Use of population,
 - Use of multiple, stochastic search operators
 - Especially variation operators with arity >1
 - Stochastic selection
- Question: what is the neighbourhood in an EA?

72

72